

Department of Technology, SPPU

Project - Low Level Design

On

Deepfake Face Detection

Using Deep learning

Team members:

Vivek Deshmukh - Roll No: BT24DSL38

Industry Mentor:

Guided by:
Prof. Manisha Bharti

Table of Contents

1. Introduction.....	3
1.1. Scope of the document.....	3
1.2. Intended audience.....	3
1.3. System overview.....	3
2. Low Level System Design.....	4
2.1. Sequence Diagram.....	4
2.2. Navigation Flow/UI Implementation.....	4
2.3. Screen Validations, Defaults and Attributes.....	5
2.4. Client-Side Validation Implementation.....	5
2.5. Server-Side Validation Implementation.....	5
2.6. Components Design Implementation.....	5
2.7. Configurations/Settings.....	7
2.8. Interfaces to other components.....	7
3. Data Design.....	8
3.1. List of Key Schemas/Tables in dataset.....	8
3.2. Details of access levels on key tables in scope.....	8
3.3. Key design considerations in data design.....	9
4. Unit Testing.....	10
5. Key notes.....	12
6. Reference.....	13

1.Introduction

1.1 Scope of the Document

This Low Level Design (LLD) document provides a detailed technical specification of the Deepfake Face Detection system. It describes the internal design of each component, the sequence of operations, the data structures used, validation logic, model architecture decisions, and the deployment interface. The document serves as a blueprint for implementation and a reference for future maintenance or extension of the system.

1.2 Intended Audience

This document is intended for the following stakeholders:

- Developers implementing or extending the deepfake detection pipeline
- Academic evaluators and project guides reviewing the system design
- Data scientists working on model improvements or retraining
- Testers designing unit and integration test cases for the system

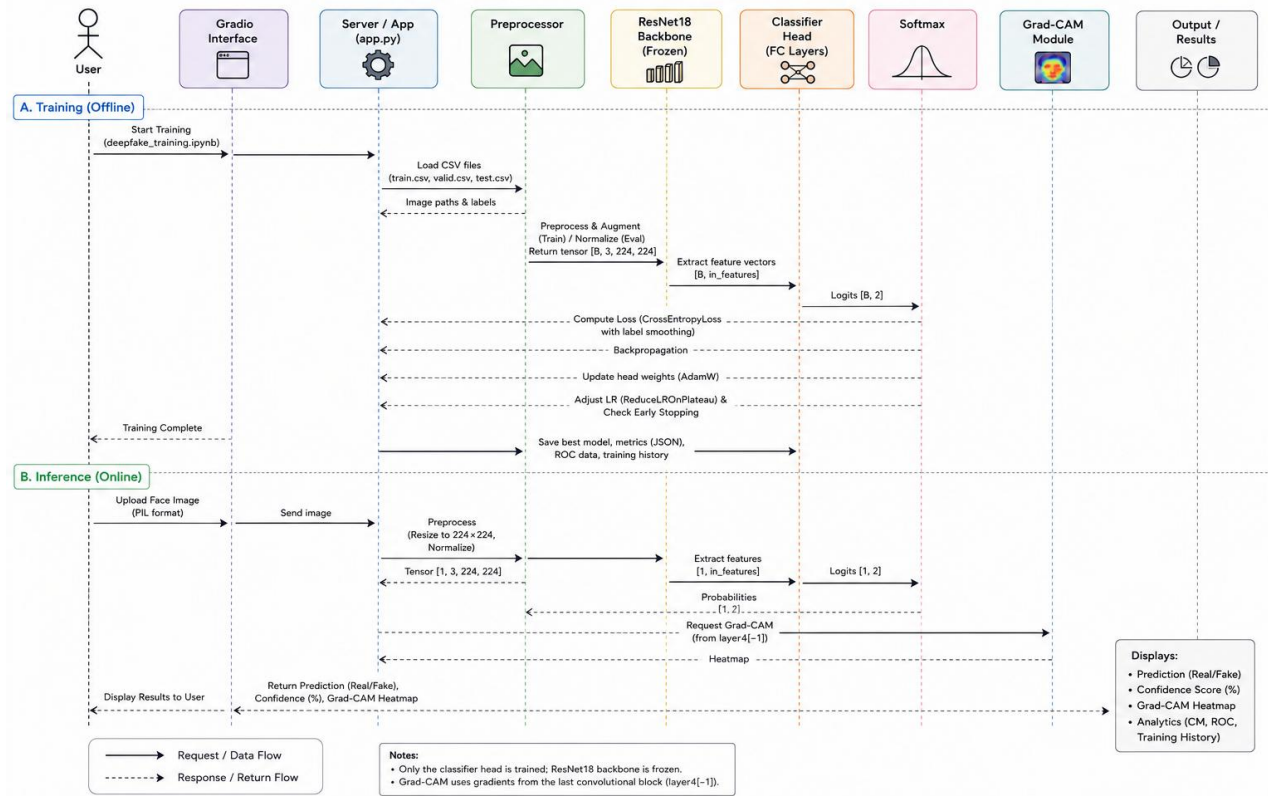
1.3 System Overview

The Deepfake Face Detection system is a binary image classification application that determines whether a given face image is real or AI-generated (fake). It is built using PyTorch with a transfer-learned ResNet18 backbone and deployed as an interactive Gradio web application. The system incorporates Grad-CAM-based visual explainability to highlight the facial regions that most influence each prediction, and provides a descriptive analytics dashboard covering model performance metrics.

2.Low Level System Design

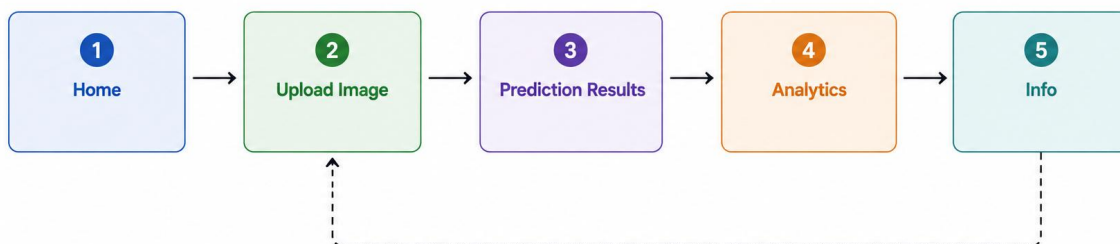
2.1 Sequence Diagram

The following describes the end-to-end sequence of operations from user interaction to prediction output:



2.2 Navigation Flow / UI Implementation

The Gradio application (app.py) exposes a single-page interface with two logical sections:



2.3 Screen Validations, Defaults and Attributes

The following input validations and defaults are applied at the UI level:

- Input type: image files only (JPEG, PNG); non-image uploads are rejected by Gradio
- Image is automatically resized to 224×224 before inference — no manual resizing required from user
- Default state: inference panel shows placeholder text until an image is uploaded
- Confidence score displayed as a percentage rounded to two decimal places
- Grad-CAM heatmap is always generated alongside the prediction — it cannot be disabled

2.4 Client-Side Validation Implementation

- Client-side validation is handled by the Gradio framework itself:
- File type enforcement: Gradio's image component accepts only valid image formats
- Empty submission guard: the predict button remains inactive until an image is uploaded
- Image preview: uploaded image is displayed immediately for user confirmation before inference

2.5 Server-Side Validation Implementation

Server-side validation is performed inside the predict() function in app.py:

- Input type check: verifies the received object is a valid PIL Image before processing
- Preprocessing guard: image is converted to RGB to handle greyscale or RGBA inputs
- Model state check: checkpoint is loaded once at startup; inference fails gracefully if the checkpoint file is missing
- Softmax assertion: output probabilities are validated to sum to 1.0 before returning the result

2.6 Components Design Implementation

The system is composed of the following independently designed components:

- **Dataset Loader Component:**
Reads image paths and labels from CSV files and creates PyTorch Dataset objects.
- **Preprocessing Component:**
Applies augmentation (training) and normalization (validation/testing) to input images.
- **Model Component (ResNet18):**
Uses a pretrained ResNet18 as a frozen feature extractor trained on ImageNet.
- **Classifier Component:**
Implements custom fully connected layers (features → 256 → 64 → 2) with ReLU and Dropout (0.3).

- **Training Component:**
Handles training using AdamW optimizer, ReduceLROnPlateau scheduler, early stopping, and checkpoint saving.
- **Grad-CAM Component:**
Generates heatmaps by extracting gradients from the final convolutional layer (layer4[-1]).
- **Output Component (Gradio App):**
Loads the trained model and provides predictions with confidence scores and visual outputs.
- **Analytics Component:**
Displays confusion matrix, ROC curve, and training history for performance analysis.

2.7 Configuration/Settings :-

The system uses predefined configurations to ensure consistent performance during execution:

- **Model Configuration:**
Uses ResNet18 as the backbone model for feature extraction and classification.
- **Input Settings:**
Accepts images resized to 224×224 for model compatibility.
- **Training Settings:**
Uses batch size of 32, trained for 10 epochs with early stopping (patience = 3).
- **Optimizer Settings:**
AdamW optimizer with learning rate 3×10^{-4} and weight decay 1×10^{-4} .
- **Regularization Settings:**
Applies dropout (0.3) and label smoothing (0.05) to improve generalization.
- **Backbone Configuration:**
Pretrained ResNet18 backbone remains frozen during training.
- **Reproducibility Settings:**
Seed value set to 42 to ensure consistent results.
- **System Settings:**
Number of workers set to 0 for data loading in the main process.

2.8 Interfaces to Other Components

The components interact through the following interfaces:

- Dataset Loader → Training Pipeline: PyTorch DataLoader objects with batched tensors and integer labels
- Training Pipeline → Artifacts: saves best_model.pt, config.json, history.json, metrics JSONs to artifacts/
- Gradio App → Model: loads best_model.pt via torch.load(); wraps in eval mode for inference
- Gradio App → Grad-CAM: registers forward/backward hooks on model.layer4[-1] during each inference call
- Gradio App → Analytics: reads best_model_roc_curve.json and history.json to render charts at startup

3.Data Design

3.1 List of Key Schemas / Tables in Dataset

This project uses file-system-based data storage rather than a relational database. The key data structures are CSV index files and image directories:

File / Structure	Type	Description
train.csv	CSV Index	100,000 rows — columns: path (relative image path), label (real/fake string)
valid.csv	CSV Index	20,000 rows — same schema as train.csv; used for validation during training
test.csv	CSV Index	20,000 rows — same schema as train.csv; used for final evaluation only
real_vs_fake/real-vs-fake/train/	Image Directory	100,000 JPEG images split into real/ and fake/ subdirectories
real_vs_fake/real-vs-fake/valid/	Image Directory	20,000 JPEG images split into real/ and fake/ subdirectories
real_vs_fake/real-vs-fake/test/	Image Directory	20,000 JPEG images split into real/ and fake/ subdirectories

3.2 Details of Access Levels on Key Data in Scope

Data / File	Training	Inference	Notes
train.csv / train images	Read	None	Accessed only during training pipeline
valid.csv / val images	Read	None	Used for checkpoint selection during training
test.csv / test images	Read	None	Accessed once for final evaluation
User-uploaded image (app)	None	Read (temp)	Held in memory during inference; not persisted to disk

3.3 Key Design Considerations in Data Design

1. Label encoding: fake $\rightarrow$ 0, real $\rightarrow$ 1 (integer labels used throughout; string labels in CSV are converted at load time)
2. Class balance: train, validation, and test splits are perfectly balanced (50% real, 50% fake) to avoid class-bias during training and evaluation
3. No data leakage: train, validation, and test sets are strictly partitioned; no image appears in more than one split
4. Normalisation: all images are normalised using ImageNet mean [0.485, 0.456, 0.406] and std [0.229, 0.224, 0.225] for compatibility with the pretrained ResNet18 backbone
5. Augmentation scope: data augmentation is applied only to training images; validation and test images use deterministic transforms.

4. Unit Testing

Unit testing is performed to verify the functionality of individual components of the Deepfake Detection System. Each module is tested independently to ensure correct behavior before integrating the complete system.

4.1. Modules Tested

1. Dataset Loader Module

- Tested loading of CSV files with image paths and labels
- Verified correct mapping of labels (fake = 0, real = 1)
- Ensured handling of missing or invalid file paths

2. Preprocessing Module

- Checked image resizing to 224×224 and normalization
- Verified consistency of output tensor shape [3, 224, 224]
- Tested transformations for both training (augmented) and evaluation (deterministic)

3. Model (ResNet18) Module

- Verified successful loading of pretrained ResNet18 backbone
- Tested forward pass using sample and dummy inputs
- Ensured correct feature extraction for classifier input

4. Classifier Module

- Checked output logits shape [batch_size, 2]
- Verified correct classification of real and fake images
- Ensured dropout and activations function properly

5. Training Pipeline Module

- Tested training on a small dataset subset
- Verified loss reduction over epochs
- Ensured checkpoint saving in artifacts/ directory

6. Grad-CAM Module

- Generated heatmaps for sample images
- Verified heatmap dimensions match input image
- Ensured values are normalized between [0, 1]

7. Output Module (Gradio App)

- Tested image upload and prediction generation
- Verified confidence scores and class labels (Real/Fake)
- Checked visualization outputs including heatmaps and analytics

5.Key Note

- The ResNet18 backbone is fully frozen during training in the current setup. Only the custom classifier head (in_features $\rightarrow$ 256 $\rightarrow$ 64 $\rightarrow$ 2) is trained. This is intentional to reduce training cost and avoid overfitting on this dataset size.
- Label smoothing (0.05) is applied to CrossEntropyLoss to reduce overconfidence in predictions and improve calibration.
- The Grad-CAM target layer is hardcoded to model.layer4[-1] (the last residual block of ResNet18). This is the deepest convolutional layer before global average pooling and provides the most semantically meaningful activation maps.
- The dataset split is perfectly balanced (50/50 real vs fake) across all three sets. Standard accuracy is therefore a reliable metric and no class-weighted loss is required.
- The model achieves ~83.76% test accuracy and 0.916 ROC-AUC with the frozen backbone. Partially unfreezing the backbone in future iterations is expected to improve performance.
- All randomness is seeded (seed=42) for reproducibility of training results.

6. References

- [1] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. IEEE CVPR 2016.
- [2] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. IEEE ICCV 2017.
- [3] Karras, T., Laine, S., & Aila, T. (2019). A Style-Based Generator Architecture for Generative Adversarial Networks. IEEE CVPR 2019.
- [4] Rossler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., & Nießner, M. (2019). FaceForensics++: Learning to Detect Manipulated Facial Images. IEEE ICCV 2019.
- [5] Tolosana, R., Vera-Rodriguez, R., Fierrez, J., Morales, A., & Ortega-Garcia, J. (2020). Deepfakes and Beyond: A Survey of Face Manipulation and Fake Detection. Information Fusion, 64, 131–148.
- [6] PyTorch Documentation. <https://pytorch.org/docs/stable/index.html>
- [7] Gradio Documentation. <https://www.gradio.app/docs>
- [8] Dataset: xhlulu. (2020). 140k Real and Fake Faces. Kaggle. <https://www.kaggle.com/datasets/xhlulu/140k-real-and-fake-faces>